FastAPI Setup and First Endpoints: Step-by-Step Guide

Beginner tutorial | Python 3.9+ | Estimated time: 45 minutes

1. What You Will Build

By the end of this guide you will have a working web API, built with FastAPI, that manages a small book catalog. You will create eight endpoints and see each of the core ideas behind FastAPI in action:

Method	URL	Concept
GET	`/`	The simplest possible endpoint
GET	`/hello/{name}`	Path parameters
GET	`/books`	Returning a list of data
GET	`/books/search?author=`	One query parameter
GET	`/books/query?category=&min_price=`	Several query parameters
POST	`/books`	Request body validation with Pydantic
GET	`/books/{book_id}`	Path parameter, type validation, 404 errors
DELETE	`/books/{book_id}`	Deleting a resource

2. Key Terms (Read This First)

- **API (Application Programming Interface):** a way for programs to talk to each other over the web, usually by exchanging JSON.
- **Endpoint:** a URL plus an HTTP method (for example `GET /books`) that does one specific job.
- **FastAPI:** the Python framework where you define your endpoints.
- **Uvicorn:** the server that actually runs your FastAPI application.

- **Pydantic:** the library FastAPI uses to validate incoming data.
- **JSON:** the text format APIs use to send data, for example `{"title": "Clean Code"}`.

> Think of it this way: FastAPI is the restaurant menu and kitchen logic, Uvicorn is the building that is open for customers.

3. Prerequisites

Before you start, confirm the following.

1. Python 3.9 or newer is installed. Check with:

```
python --version
```

On some systems the command is `python3 --version`. If you see `Python 3.9.x` or higher, you are ready.

2. You have a code editor (VS Code is a good choice) and a terminal.
3. You have a web browser.

Step 1: Create a Project Folder

Open your terminal and run:

```
mkdir fastapi-demo
cd fastapi-demo
```

What this does: creates an empty folder for the project and moves you into it. Every command from now on is run from inside this folder.

Step 2: Create a Virtual Environment

A virtual environment is a private box of Python packages for this project only. It stops different projects from interfering with each other.

```
python -m venv venv
```

What this does: creates a folder called `venv` that holds an isolated copy of Python.

Step 3: Activate the Virtual Environment

Choose the command for your operating system.

macOS and Linux:

```
source venv/bin/activate
```

Windows (Command Prompt):

```
venv\Scripts\activate
```

Windows (PowerShell):

```
venv\Scripts\Activate.ps1
```

How to know it worked: your terminal prompt now starts with `(venv)`.

> If PowerShell blocks the script, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again.

Step 4: Install FastAPI and Uvicorn

```
pip install fastapi "uvicorn[standard]"
```

What this does: downloads FastAPI, Uvicorn, and the libraries they depend on (including Pydantic).

Verify the install:

```
pip list
```

You should see `fastapi`, `uvicorn`, and `pydantic` in the list.

Optional: save your dependencies so others can reproduce your setup.

```
pip freeze > requirements.txt
```

Anyone can then install everything with `pip install -r requirements.txt`.

Step 5: Create main.py with the Home Endpoint

Create a new file named `main.py` in the `fastapi-demo` folder and add:

```
from fastapi import FastAPI

app = FastAPI()


# ------------------------------------------------------------
# 1. HOME ENDPOINT
# URL: http://localhost:8000/
# ------------------------------------------------------------

@app.get("/")
def home():
    return {"message": "Hello, FastAPI!"}
```

Line by line:

- `from fastapi import FastAPI` imports the framework.
- `app = FastAPI()` creates your application object. Everything attaches to `app`.
- `@app.get("/")` is a *decorator*. It says: "when someone sends a GET request to `/`, run the function below."
- `def home():` is a normal Python function.
- `return {...}` returns a dictionary. FastAPI converts it to JSON automatically.

Step 6: Run the Server

```
uvicorn main:app --reload
```

- `main` is the file `main.py` (without `.py`).
- `app` is the object named `app` inside that file.
- `--reload` restarts the server whenever you save a change. Use it during development only.

You should see output similar to:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process
INFO:     Application startup complete.
```

Leave this terminal running. Press **Ctrl+C** to stop the server later.

Step 7: Call the Endpoint and Open the Auto-Generated Docs

Visit `http://localhost:8000/` in your browser. You should see:

```
{"message": "Hello, FastAPI!"}
```

Now open the interactive documentation that FastAPI builds from your code:

- **Swagger UI:** `http://localhost:8000/docs`
- **ReDoc:** `http://localhost:8000/redoc`

In Swagger UI, expand `GET /`, click **Try it out**, then **Execute**. You will use this page to test every endpoint in the rest of this guide.

Step 8: Add a Path Parameter (Greet a User)

A path parameter is a variable part of the URL. Add this to the bottom of `main.py`:

```
# ------------------------------------------------------------
# 2. PATH PARAMETER - GREET USER
# URL: http://localhost:8000/hello/Ashu
# ------------------------------------------------------------

@app.get("/hello/{name}")
def say_hello(name: str):
    return {"message": f"Hello, {name}!"}
```

Save the file. The server reloads by itself.

Test it: visit `http://localhost:8000/hello/Ashu`. You should see `{"message": "Hello, Ashu!"}`. Change the name in the URL and refresh.

How it works: the `{name}` in the URL matches the `name` argument of the function. FastAPI passes the value in for you.

Step 9: Add Sample Data and Return All Books

Add a Python list that acts as a simple in-memory database, then an endpoint that returns it. Append this to `main.py`:

```
# ------------------------------------------------------------
# 3. SAMPLE BOOK DATA
# ------------------------------------------------------------

books = [
    {
        "id": 1,
        "title": "Clean Code",
        "author": "Robert C. Martin",
        "category": "Programming",
        "min_price": 500.0
    },
    {
        "id": 2,
        "title": "The Pragmatic Programmer",
        "author": "Andrew Hunt",
        "category": "Programming",
        "min_price": 600.0
    }
]


# ------------------------------------------------------------
# 4. GET ALL BOOKS
# URL: http://localhost:8000/books
# ------------------------------------------------------------

@app.get("/books")
def list_books():
    return books
```

Test it: visit `http://localhost:8000/books`. You should see both books as a JSON list.

Note: the data lives in memory, so every change is lost when the server restarts. That is expected for this demo.

Step 10: Search by Author (Query Parameter)

A query parameter is the part of the URL after the `?`. Append:

```
# ------------------------------------------------------------
# 5. SEARCH BOOKS BY AUTHOR - QUERY PARAMETER
# URL: http://localhost:8000/books/search?author=Robert
# ------------------------------------------------------------

@app.get("/books/search")
def search_books(author: str):

    return [
        b for b in books
        if author.lower() in b["author"].lower()
    ]
```

Test it:

URL	Result
`/books/search?author=Robert`	Clean Code
`/books/search?author=hunt`	The Pragmatic Programmer
`/books/search`	422 error: `author` is required

How it works:

- `author` is not part of the path, so FastAPI treats it as a query parameter.
- It has no default value, so it is **required**. Leaving it out gives an automatic 422 error.
- `.lower()` on both sides makes the search case-insensitive, and `in` matches part of the name.

Step 11: Filter by Category and Minimum Price (Multiple Query Parameters)

You can combine several query parameters. They are separated by `&`. Append:

```
# ------------------------------------------------------------
# 6. QUERY PARAMETERS - CATEGORY AND MINIMUM PRICE
# URL: http://localhost:8000/books/query?category=Programming&min_price=450
# ------------------------------------------------------------

@app.get("/books/query")
def get_books(category: str, min_price: float):

    result = [
        b for b in books
        if b["category"].lower() == category.lower()
        and b["min_price"] >= min_price
    ]

    return result
```

Test it:

URL	Result
`/books/query?category=Programming&min_price=450`	Both books
`/books/query?category=Programming&min_price=550`	Only The Pragmatic Programmer
`/books/query?category=Programming&min_price=abc`	422 error: `min_price` must be a number

How it works: `min_price: float` tells FastAPI to convert the text from the URL into a decimal number and reject anything that is not one. The filter keeps books whose category matches and whose `min_price` is at least the value you passed.

> **Important:** `/books/search` and `/books/query` are placed *before* the `/books/{book_id}` endpoint you will add in Step 14. FastAPI checks routes from top to bottom, so if `/books/{book_id}` came first, the word `search` would be treated as a book id and fail.

Step 12: Create the BookIn Model in book.py

Before accepting data from clients, describe what a valid book looks like. Create a **new file** named `book.py` in the same folder as `main.py`:

```
from pydantic import BaseModel


class BookIn(BaseModel):
    title: str
    author: str
    category: str
    min_price: float
```

How it works:

- `BaseModel` comes from Pydantic. A class built on it describes the shape of your data.
- Each line is a field with a name and a type. All four fields are required.
- Keeping the model in its own file keeps `main.py` short and makes the model reusable.

Now import it. Add this line near the top of `main.py`, under the first import:

```
from fastapi import FastAPI
from book import BookIn
```

Your project folder now looks like this:

```
fastapi-demo/
├── venv/
├── main.py
└── book.py
```

Step 13: Add a Book with POST

So far you only *read* data. Now let clients *send* data. Append to `main.py`:

```
# ------------------------------------------------------------
# 7. ADD NEW BOOK - POST REQUEST
# URL: http://localhost:8000/books
# Method: POST
# ------------------------------------------------------------

@app.post("/books", status_code=201)
def add_book(book: BookIn):

    new_book = {
        "id": len(books) + 1,
        **book.model_dump()
    }

    books.append(new_book)

    return new_book
```

How it works:

- Because `book` is typed as `BookIn`, FastAPI reads the JSON body, validates it, and gives you a ready-to-use object.
- `status_code=201` means "Created", the correct status for a successful POST.
- `book.model_dump()` converts the Pydantic object into a dictionary.
- `**` unpacks that dictionary into `new_book`, next to the new `id`.
- `len(books) + 1` gives the next id.

Test it in Swagger UI (`http://localhost:8000/docs`):

1. Expand `POST /books` and click **Try it out**.
2. Replace the body with:

```
{
  "title": "Refactoring",
  "author": "Martin Fowler",
  "category": "Programming",
  "min_price": 700.0
}
```

3. Click **Execute**. You should get status **201** and the new book with `id` 3.
4. Call `GET /books` to confirm it was added.

Test it with curl:

```
curl -X POST http://localhost:8000/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Refactoring", "author": "Martin Fowler", "category":
"Programming", "min_price": 700.0}'
```

Test validation: send a body without `author`, or with `"min_price": "cheap"`. FastAPI returns status **422** and names the field that is wrong.

Step 14: Get One Book by ID and Handle Errors

First, update the first line of `main.py` to import `HTTPException`:

```
from fastapi import FastAPI, HTTPException
from book import BookIn
```

Then append:

```
# ------------------------------------------------------------
# 8. GET BOOK BY ID - PATH PARAMETER
# URL: http://localhost:8000/books/1
# ------------------------------------------------------------

@app.get("/books/{book_id}")
def get_book(book_id: int):

    for book in books:
        if book["id"] == book_id:
            return book

    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )
```

Test three cases:

URL	Result
`/books/1`	The book with id 1 (status 200)

URL	Result
`/books/99`	`{"detail": "Book not found"}` (status 404)
`/books/abc`	Automatic validation error (status 422)

Key ideas:

- The type hint `book_id: int` is not just documentation. FastAPI checks it and rejects bad input for you.
- `raise HTTPException(...)` stops the function and sends a proper error response to the client.
- This endpoint sits *below* `/books/search` and `/books/query` on purpose (see the note in Step 11).

Step 15: Delete a Book with DELETE

Append the last endpoint:

```
# ------------------------------------------------------------
# 9. DELETE BOOK - DELETE REQUEST
# URL: http://localhost:8000/books/1
# Method: DELETE
# ------------------------------------------------------------

@app.delete("/books/{book_id}")
def delete_book(book_id: int):

    for book in books:

        if book["id"] == book_id:

            books.remove(book)

            return {
                "message": "Book deleted successfully",
                "deleted_book": book
            }
```

```
    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )
```

Test it in Swagger UI:

1. Expand `DELETE /books/{book_id}`, click **Try it out**, enter `1`, and click **Execute**.
2. You should see `"Book deleted successfully"` and the deleted book.
3. Call `GET /books/1`. It now returns **404**.
4. Call `DELETE /books/99`. It returns **404** because there is nothing to delete.

How it works: the same path `/books/{book_id}` is used by both GET and DELETE. The HTTP *method* decides which function runs.

> **Watch out:** the POST endpoint builds new ids with `len(books) + 1`. After a delete, that can repeat an existing id (delete book 1 from a list of two, and the next book also gets id 2). It is fine for a demo, and a good talking point for why real applications use a database to generate ids.

Step 16: Review the Complete Code

Your finished project has two files. Compare yours against these.

book.py

```
from pydantic import BaseModel


class BookIn(BaseModel):
    title: str
    author: str
    category: str
    min_price: float
```

main.py

```
from fastapi import FastAPI, HTTPException
from book import BookIn

app = FastAPI()
```

```
# ------------------------------------------------------------
# 1. HOME ENDPOINT
# URL: http://localhost:8000/
# ------------------------------------------------------------

@app.get("/")
def home():
    return {"message": "Hello, FastAPI!"}


# ------------------------------------------------------------
# 2. PATH PARAMETER - GREET USER
# URL: http://localhost:8000/hello/Ashu
# ------------------------------------------------------------

@app.get("/hello/{name}")
def say_hello(name: str):
    return {"message": f"Hello, {name}!"}


# ------------------------------------------------------------
# 3. SAMPLE BOOK DATA
# ------------------------------------------------------------

books = [
    {
        "id": 1,
        "title": "Clean Code",
        "author": "Robert C. Martin",
        "category": "Programming",
        "min_price": 500.0
    },
    {
        "id": 2,
        "title": "The Pragmatic Programmer",
        "author": "Andrew Hunt",
        "category": "Programming",
        "min_price": 600.0
    }
]
```

```
# ------------------------------------------------------------
# 4. GET ALL BOOKS
# URL: http://localhost:8000/books
# ------------------------------------------------------------

@app.get("/books")
def list_books():
    return books


# ------------------------------------------------------------
# 5. SEARCH BOOKS BY AUTHOR - QUERY PARAMETER
# URL: http://localhost:8000/books/search?author=Robert
# ------------------------------------------------------------

@app.get("/books/search")
def search_books(author: str):

    return [
        b for b in books
        if author.lower() in b["author"].lower()
    ]


# ------------------------------------------------------------
# 6. QUERY PARAMETERS - CATEGORY AND MINIMUM PRICE
# URL: http://localhost:8000/books/query?category=Programming&min_price=450
# ------------------------------------------------------------

@app.get("/books/query")
def get_books(category: str, min_price: float):

    result = [
        b for b in books
        if b["category"].lower() == category.lower()
        and b["min_price"] >= min_price
    ]

    return result


# ------------------------------------------------------------
# 7. ADD NEW BOOK - POST REQUEST
```

```
# URL: http://localhost:8000/books
# Method: POST
# ------------------------------------------------------------

@app.post("/books", status_code=201)
def add_book(book: BookIn):

    new_book = {
        "id": len(books) + 1,
        **book.model_dump()
    }

    books.append(new_book)

    return new_book


# ------------------------------------------------------------
# 8. GET BOOK BY ID - PATH PARAMETER
# URL: http://localhost:8000/books/1
# ------------------------------------------------------------

@app.get("/books/{book_id}")
def get_book(book_id: int):

    for book in books:
        if book["id"] == book_id:
            return book

    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )


# ------------------------------------------------------------
# 9. DELETE BOOK - DELETE REQUEST
# URL: http://localhost:8000/books/1
# Method: DELETE
# ------------------------------------------------------------

@app.delete("/books/{book_id}")
def delete_book(book_id: int):
```

```
    for book in books:

        if book["id"] == book_id:

            books.remove(book)

            return {
                "message": "Book deleted successfully",
                "deleted_book": book
            }

    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )
```

Endpoint checklist:

Step	Method	URL	Expected result
5	GET	`/`	Hello message
8	GET	`/hello/Ashu`	Greeting with the name
9	GET	`/books`	All books
10	GET	`/books/search?author=Robert`	Matching books
11	GET	`/books/query?category=Programming&min_price=450`	Filtered books
13	POST	`/books`	201 and the new book
14	GET	`/books/1`	One book, or 404
15	DELETE	`/books/1`	Success message, or 404

17. Quick Reference: HTTP Status Codes Used

Code	Meaning	Where you saw it
200	OK	Successful GET requests
201	Created	Successful POST
404	Not Found	`/books/99`
422	Unprocessable Entity	Invalid type or missing field

18. Troubleshooting

Problem	Likely cause	Fix
`uvicorn: command not found`	Virtual environment not active	Re-run the activation command from Step 3
`Error loading ASGI app`	Wrong folder or file name	Run the command from the folder that contains `main.py`
`ModuleNotFoundError: fastapi`	Package not installed in this environment	Activate `venv`, then run `pip install fastapi "uvicorn[standard]"`
`ModuleNotFoundError: No module named 'book'`	`book.py` is missing, misspelled, or in another folder	Put `book.py` in the same folder as `main.py` (Step 12)
`/books/search` returns a 422 error about `book_id`	`/books/{book_id}` is defined above `/books/search`	Move the search and query endpoints above the `{book_id}` endpoints
`Address already in use`	Port 8000 is taken	Use `uvicorn main:app --reload --port 8001`
Changes not showing	Server not reloading	Save the file and check the terminal for errors; restart Uvicorn

Problem	Likely cause	Fix
Books disappear after restart	Data is stored in memory	Expected behavior in this demo

19. Practice Exercises

1. **New field:** add an optional `year: int | None = None` field to `BookIn` and confirm it appears in `/docs`.
2. **Count:** add `GET /books/count` that returns the total number of books. Hint: where you place this route matters, because `/books/{book_id}` could match `count` first.
3. **Safer ids:** delete a book, then add one. Notice the id problem from Step 15, then fix it by generating ids from the highest existing id instead of `len(books) + 1`.
4. **Optional filter:** make `category` optional in `/books/query` (`category: str | None = None`) so the endpoint can filter by price alone.

20. Where to Go Next

- Replace the in-memory list with a real database (SQLite with SQLAlchemy is a gentle start).
- Add a `PUT` endpoint to update a book.
- Organize code into separate files with `APIRouter`.
- Write automated tests with `TestClient`.
- Read the official FastAPI tutorial at fastapi.tiangolo.com.